

Rapport TP2

LOG8235 - Agents intelligents pour jeux vidéo

Equipe

Baptiste Leducq- 2486057

Guillaume Lesourd - 2488094

Mauricio Medina - 1537659

Noah Mesple - 2487453

1. Génération du navmesh



Nous avons utilisé 2 navmesh construits par 2 volumes, un pour la rivière, un pour le terrain.

2. Déplacement du joueur et affichage des chemins

ASoftDesignTrainingPlayerController:

```

void ASoftDesignTrainingPlayerController::MoveCharacter()
{
    // TODO : find the position of the mouse in the world
    // And move the agent to this position IF possible
    // Validate you can move through m_CanMoveCharacter

    FVector WorldLocation;
    FVector WorldDirection;

    DeprojectMousePositionToWorld(WorldLocation, WorldDirection);

    FHitResult ResultOut;
    FCollisionObjectQueryParams QueryParams;
    QueryParams.AddObjectTypesToQuery(ECC_WorldStatic);

    FVector StartLocation = WorldLocation;
    FVector EndLocation = WorldLocation + 10000.f * WorldDirection;

    bool Success = GetWorld()->LineTraceSingleByObjectType(ResultOut,
        StartLocation,
        EndLocation,
        QueryParams);

    if (Success)
    {
        if (m_CanMoveCharacter)
        {
            UNavigationSystemV1* NavSys = FNavigationSystem::GetCurrent<UNavigationSystemV1>(GetWorld());

```

DeprojectMousePositionToWorld nous donne la position de la souris et la direction pour lancer un rayon (LineTraceSingleByObjectType) et projeter l'intersection du rayon avec le terrain dans le navmesh.

```

        if (NavSys->ProjectPointToNavigation(ResultOut.ImpactPoint, Projected))
    }
}

```

Suit à cela on calcule FindPathToLocationSynchronously pour avoir le point du chemin pour après le designer dans DebugDrawCalculatedPath

```

158     UNavigationPath* NavPath = NavSys->FindPathToLocationSynchronously(
159         GetWorld(),
160         MyChar->GetActorLocation(),
161         Projected.Location,
162         MyChar
163     );
164
165     if (NavPath)
166     {
167         DebugDrawCalculatedPath(NavPath);
168         UAIBlueprintHelperLibrary::SimpleMoveToLocation(this, Projected.Location);
169     }
170

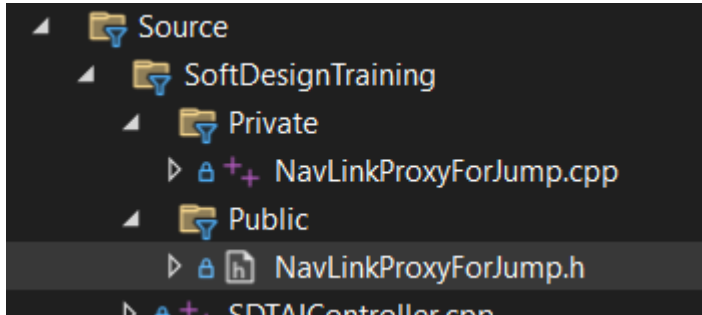
```

3. Parcours de chemin

Pour que les piétons se déplacent, nous avons rempli les todos dans le fichier SDTAIController.cpp

4. Rajout de navlink

On a créé une class NavLinkProxyForJump.



On s'est abonné à l'événement, OnSmartLinkReached. Pour être notifié quand le player arrive dans le point initial du navlink pour se préparer pour le saut.

```
void ANavLinkProxyForJump::BeginPlay()
{
    Super::BeginPlay();

    OnSmartLinkReached.AddDynamic(this, &ANavLinkProxyForJump::SmartLinkReachedForJump);
}
```

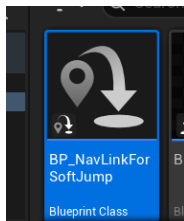
```

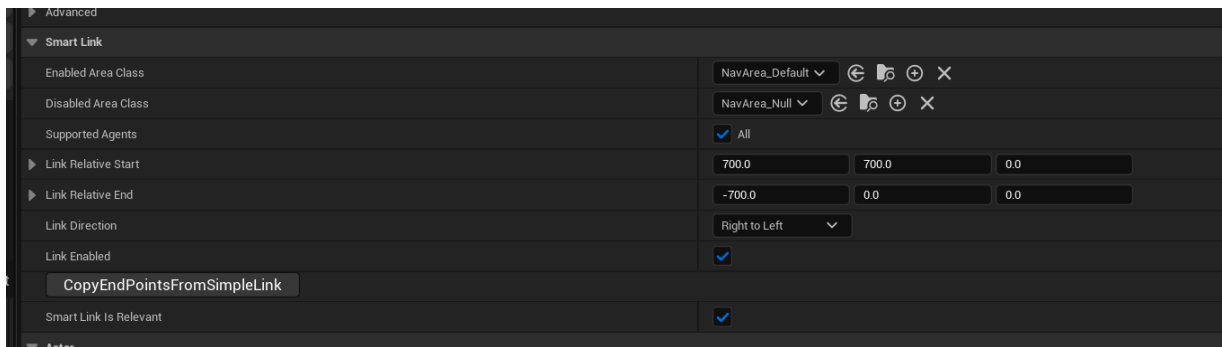
oftDesignTraining
    ↓ ANavLinkProxyForJump
16 void ANavLinkProxyForJump::SmartLinkReachedForJump(AActor* MovingActor, const FVector& DestinationPoint)
17 {
18     ASoftDesignTrainingMainCharacter* SoftCharacter = Cast<ASoftDesignTrainingMainCharacter>(MovingActor);
19
20     if (SoftCharacter)
21     {
22         MaxWalkSpeed = SoftCharacter->GetCharacterMovement()->MaxWalkSpeed;
23
24         SoftCharacter->GetCharacterMovement()->MaxWalkSpeed = 0.0f;
25         SoftCharacter->m_IsAtJumpPosition = true;
26
27         FTimerHandle ResumeHandle;
28         GetWorldTimerManager().SetTimer(
29             ResumeHandle,
30             [this, SoftCharacter, DestinationPoint]()
31             {
32                 SoftCharacter->GetCharacterMovement()->MaxWalkSpeed = this->MaxWalkSpeed;
33
34                 FVector Start = SoftCharacter->GetActorLocation();
35                 FVector End = DestinationPoint;
36                 FVector LaunchVelocity;
37
38                 bool bFound = UGameplayStatics::SuggestProjectileVelocity_CustomArc(
39                     this,
40                     LaunchVelocity,
41                     Start,
42                     End,
43                     0, /*= 0*/
44                     0.5 /*= 0.5f */
45                 );
46
47                 if (bFound)
48                 {
49                     SoftCharacter->LaunchCharacter(LaunchVelocity, true, true);
50                     SoftCharacter->m_IsAtJumpPosition = false;
51                 }
52                 //UE_LOG(LogTemp, Display, TEXT("Smart link jump complete - FAILD."));
53             },
54             0.3f, // delay in seconds
55             false // looping = false
56         );
57     }
58 }
59
60

```

Quand le joueur arrive dans le point de depart on met la vitesse a 0 et on informe au joueur qu'il est arrivé au point de départ pour le saut (line 25), cela va être utilisé pour les animations. On delai 0.3 secondes pour après calculé est déclenchée le saut. C'est delay permet de jouer l'animation pour le saut.

5. Parcours d'un chemin comportant des navlinks





Le smart navlink est configuré de la façon comme est montré dans l'image, on garde une seule direction pour que les AI l' utilise pas. Ça oblige le joueur à emprunter le pont pour retourner au point de départ après le saut.

Note: On utilise la logique par défaut de unreal du PathFollowingComponent, parce que le smart navlink facilite le travail en nous donnant le point de depart pour le saut.

```
21 //  
22 void USDTPathFollowingComponent::FollowPathSegment(float DeltaTime)  
23 {  
24     Super::FollowPathSegment(DeltaTime);  
25  
26     //if (!Path)  
27     //if
```

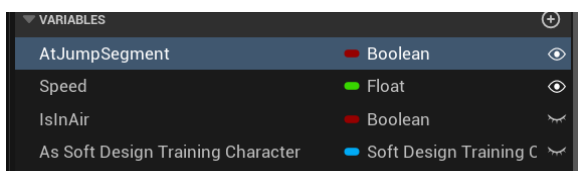
6. Génération d'un second navmesh

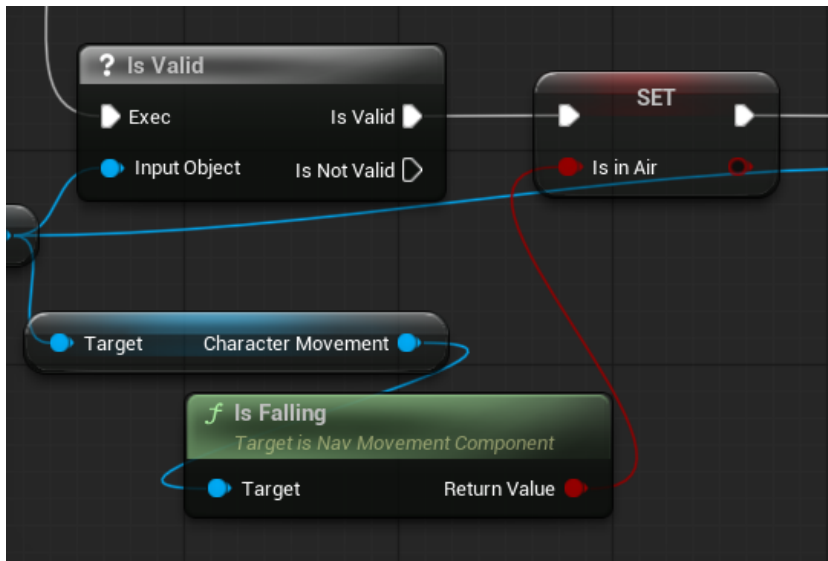
Pour générer le deuxième navmesh, nous avons ajouté une deuxième navmesh bound volume seulement au niveau du fleuve, et nous avons remonté la première bounding volume pour qu'elle n'englobe que les deux rives.

7. Déplacement des bateaux

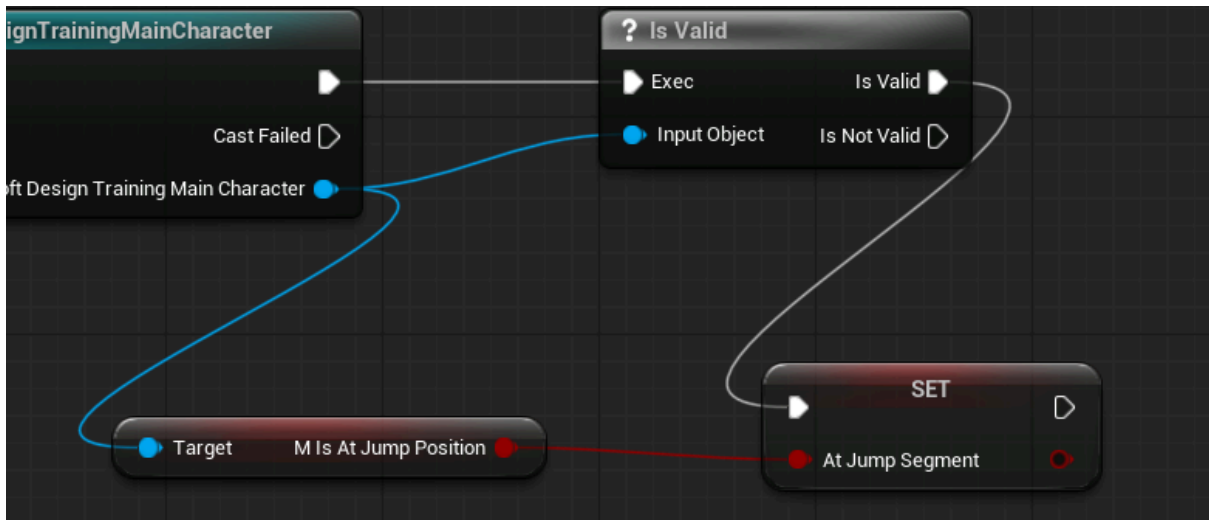
Pour que les bateaux se déplacent nous avons rempli les todos dans le fichier SDTBoatAIController.cpp.

8. Ajout d'animation de déplacement





Nous utilisons is falling du character mouvement pour mettre à jour “IsInAir”.



“IsAtJumpPosition” met à jour “AtJumpSegment” pour le transition de walk to jump start

9. Ajout d'animation de saut

Machine d'état pour sauter et macher.

